

Executive Summary

This document presents a complete set of deliverables for a Smart India Hackathon (SIH) solution, covering all stages from requirements to deployment. We outline a **Product Requirements Document (PRD)** with features (including an MVP), user stories, acceptance criteria, and prioritization. Detailed workflows and user-journeys are described for each user role. We define business and validation rules, list edge cases, and propose a robust technical architecture (frontend, backend, ML/model, data schema, APIs) with diagrams. UI/UX wireframe components and screens are enumerated. The **Test Plan** (unit, integration, end-to-end) and **Deployment/CI/CD** checklist (including security and monitoring) are specified. Data privacy and compliance considerations are noted. A detailed **Project Plan** includes milestones, a Gantt chart, resource estimates, and a risk register. We also provide sample `README.md` and contributor guide content. Tables compare technology stacks, team roles, and a prioritized feature backlog. Wherever possible we cite best-practice sources. This template should be adapted to the specific problem statement by filling in contextual details (for example, the actual features and domain data). It assumes an Agile approach, where a concise PRD aligns stakeholders on goals, much like Atlassian describes: “A product requirements document (PRD) defines the purpose, features, and behavior of a product, aligning stakeholders and guiding development” [4†L1501-L1504] .

Product Requirements Document (PRD)

Project Overview

- **Problem Statement** (from SIH): *[Describe the given SIH problem here.]*
- **Goal:** Deliver a working solution that addresses the PS by providing [high-level summary of solution].

Key Features and MVP

We list all desired features from smallest enhancements to major components. For example:

- **Must-have (MVP):**
 - **User Authentication & Authorization:** Secure login/logout, role-based access (admin vs user).
 - **Core Functionality:** The solution’s primary feature (e.g. data entry, analytics dashboard, real-time alerts) to satisfy the problem.
 - **Basic UI for Main Use Cases:** Minimal screens/views needed to demonstrate functionality.
 - **API Endpoints:** RESTful APIs for core data operations (create/read/update/delete).
 - **Database Schema:** Core tables/collections to support data requirements.
- **Should-have (Medium Priority):**
 - **Advanced Search/Filter:** If applicable, allow users to easily query or filter data.
 - **Reporting/Export:** Generate PDF/CSV reports, or charts summarizing key metrics.
 - **User Management (Admin):** Admin ability to manage users, roles, or settings.

- **Notification System:** Email/SMS or in-app notifications for important events.
- **Responsive Design:** Make UI mobile-friendly.
- **Could-have (Nice-to-have):**
 - **Dashboard Widgets:** Customizable dashboard elements or live metrics.
 - **Multi-language Support:** If relevant for wider user base.
 - **Integration with Third-party Services:** e.g. payment gateway, external data APIs.
- **Won't-have (Out of Scope):**
 - Features planned beyond initial release (e.g. secondary modules, complex analytics beyond MVP). These are noted to prevent scope creep.

This categorization follows the MoSCoW prioritization method (Must/Should/Could/Won't) 【16†L30-L39】. It ensures we focus on delivering high-value items first. The **MVP** is the smallest set of must-have features needed to make the solution functional.

User Stories and Acceptance Criteria

We capture functionality from the user's perspective. Each feature is described by one or more user stories (with acceptance criteria). For example:

- **Story 1 (User Login):** *"As a registered user, I want to log in using my email and password so that I can access my dashboard."*
 - **Acceptance Criteria:**
 - Valid credentials direct the user to the dashboard 【42†L1694-L1702】.
 - Invalid credentials show an error message (with no login).
 - After login, a session token or cookie is securely stored.
 - Users have the appropriate role (user/admin) once logged in.
- **Story 2 (Data Entry):** *"As a user, I want to submit new [data item] through a form, so that it is saved in the system."*
 - **Acceptance Criteria:**
 - Required fields must be filled; empty required fields cause inline validation errors.
 - On success, the system shows a confirmation message and the new item appears in the listing.
 - Input is sanitized and stored persistently (testable via DB or API).
- **Story 3 (Admin User Management):** *"As an admin, I want to view, add, or deactivate users so that I can control who has access."*

- **Acceptance Criteria:**

- Admin can see a user list with statuses.
- Admin can deactivate (or delete) a user; the user then cannot log in.
- Email notifications sent for password reset or activation (if applicable).

Each user story articulates the “why” and value for the user, while acceptance criteria define “what success looks like” in concrete, testable terms [42†L1713-L1721] . Following best practices, acceptance criteria are clear, unambiguous, and measurable [42†L1749-L1758] [42†L1769-L1778] . They form the **Definition of Done** for each story.

Prioritization and Backlog

All features and stories are maintained in a backlog, ordered by priority. We prioritize using **business value and urgency**: items with greatest impact go first [16†L30-L39] [45†L1-L4] . A simple table might track this:

Priority	Feature/User Story	Rationale	Status
Must	User Login / Auth	Essential for secure access.	Planned
Must	Core Data CRUD	Core problem solution functionality.	Planned
Should	Reporting Dashboard	High business value for analytics.	Defined
Should	Notification Service	Improves responsiveness.	Defined
Could	Multi-language UI	Accessibility for wider users.	Future
Won't	Advanced AI module	Out-of-scope for MVP.	Deferred

We may use timeboxing (fixed deadline) to ensure focus on **Must** and **Should** items [16†L50-L53] . The backlog is regularly groomed by the Product Owner (team lead) and aligned with the project goal.

Non-Functional Requirements (NFRs)

NFRs define system qualities and constraints, such as:

- **Performance:** The system should handle [X] concurrent users/respond within [Y] seconds for key operations.
- **Security:** Use HTTPS, store passwords hashed (e.g. bcrypt), implement OWASP Top-10 mitigations (see Security Checklist below).
- **Scalability:** Architected to scale horizontally (e.g. stateless services, containerizable) if user base grows.
- **Reliability/Availability:** 99% uptime target; use retry logic and backups for critical data.
- **Usability:** UI must be intuitive; conform to accessibility standards (e.g. WCAG) if required.
- **Maintainability:** Use clear code standards, modular architecture, automated tests to ease future work.
- **Compliance:** Follow data protection laws (e.g. GDPR) for user data (see Data Privacy section).

These NFRs act as acceptance criteria for the system's architecture and implementation.

Workflows and User Journeys

We describe how each role interacts with the system, step by step.

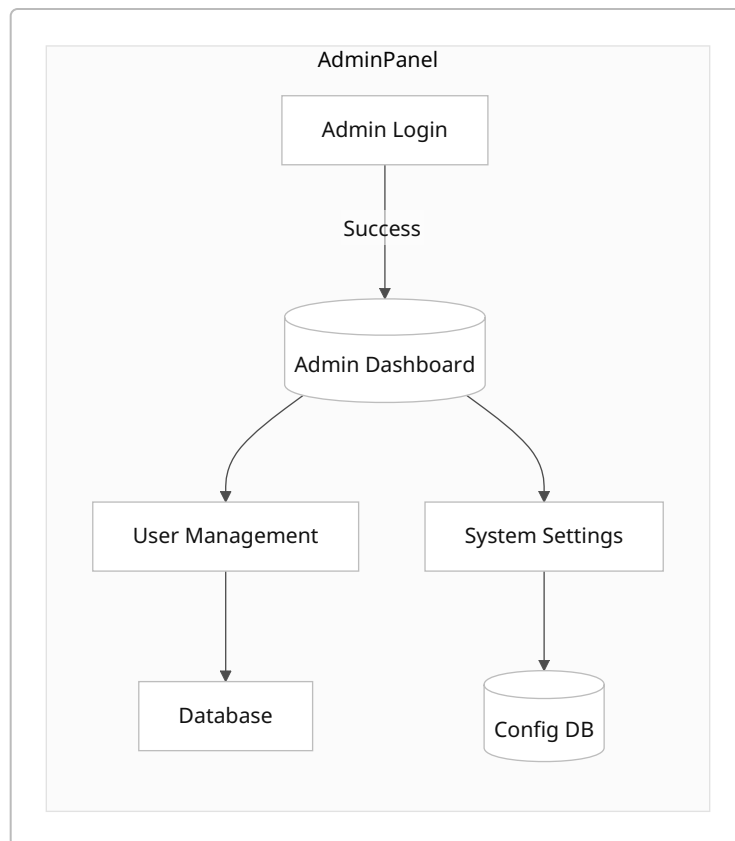
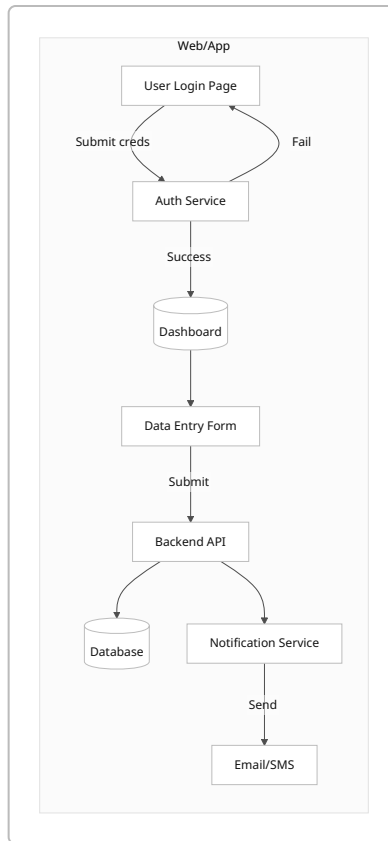
Roles and High-Level Journeys

- **End User:** The primary user of the solution (e.g. a citizen, student, patient, etc.). Typical journey:
 - **Access/Login:** Navigate to the app and log in.
 - **Dashboard/View Data:** See personalized data or status on the main screen.
 - **Perform Task:** Create or update a record (e.g. submit an application, raise a request).
 - **Receive Notification:** Get confirmation or alerts via email/in-app as needed.
- **Log Out:** Securely end session.
- **Administrator:** Manages the system and users. Typical journey:
 - **Login as Admin:** Separate admin portal or admin-level login.
 - **User Management:** View all users, assign roles, deactivate accounts.
 - **Monitor Data:** Access all records or reports.
 - **System Settings:** Configure system parameters (e.g. thresholds, messages).
- **Log Out.**
- **(Optional) Expert/Moderator:** If the PS involves content review (e.g. medical expert reviewing results), another role could review submitted items and take actions. Journey:
 - **Review Task Queue:** See pending items needing approval.
 - **Update Item:** Approve/reject or annotate records.
 - **Notify User:** Trigger notification on action taken.
- **Log Out.**

These user journeys map the high-level scenario across the system [46†L83-L91] . We also break them into user flows (each flow corresponds to one interaction):

- **User Flow: Login:** User → Login Screen → Enter credentials → Submit → Dashboard or error.
- **User Flow: Submit Data:** User → Data Entry Form → Fill in fields → Validate → Save → Confirmation.
- **User Flow: View Report:** User → Reports Page → Select filters/date → System fetches data → Display chart/table.

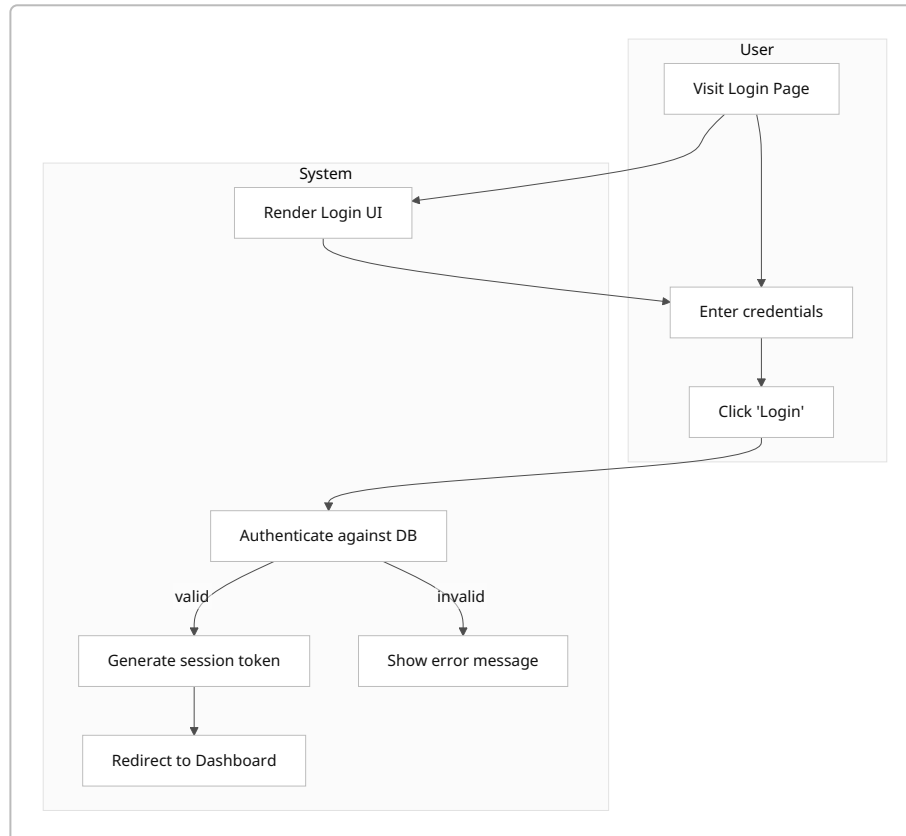
Illustrative mermaid user-flow diagrams:



These flows (level-of-detail diagrams) show each step from user action to system response. Per UX best practices, we ensure each flow is smooth and intuitive 【46†L83-L91】 【46†L123-L131】. For example, journey maps might annotate user “pain points” or emotions in long processes, but for this hackathon scope, we focus on functional steps.

Wireflows and Swimlanes

For complex interactions, we might create swimlane flowcharts. Example (Mermaid syntax for swimlane not native, but logical grouping):



Here, the “User” lane and “System” lane illustrate responsibilities in the login sequence. Such diagrams help clarify where client vs server logic sits.

Edge Cases in Workflows

We explicitly consider edge cases, e.g.:

- **Session Expiry:** If user is inactive or session times out, they are redirected to login.
- **Concurrent Edits:** If two users edit the same data, system should handle conflicts (e.g. last-write wins or locking).
- **Network Failure:** Ensure client recovers gracefully (show error, allow retry).
- **Invalid Inputs:** All forms validate client-side and server-side (e.g. numeric fields, required fields).

- **Authorization:** Prevent role escalation (e.g. a normal user should not access admin functions via URL).

These are covered both in design (UI/UX) and in rules/validations below.

Business Rules, Validation Rules, and Edge Cases

This document captures all domain/business-specific rules and input validations.

Business Rules

- **User Uniqueness:** No two users may have the same email/ID. Attempt to register an existing email returns an error.
- **Role Restrictions:** Only **Admin** can change roles or deactivate accounts.
- **Data Retention:** User data must be archived or deleted after [X] years (for compliance).
- **Report Generation:** Reports can only be generated if at least N data records exist.
- **Alerts Threshold:** e.g. if sensor data exceeds a threshold, automatically trigger an alert.

For each rule, we define the condition and outcome. These rules must be documented so developers know how to implement and testers can verify them.

Validation Rules

- **Form Inputs:** Enforce data types and formats (e.g. date fields, numeric ranges).
- **Length Limits:** Text inputs capped (e.g. description $\leq$ 500 characters).
- **Cross-field Validation:** If field A is set to X, then field B must be non-empty.
- **Injection Prevention:** Sanitize all text to prevent SQL/NoSQL injection and XSS.
- **File Uploads:** If files (images, docs) are allowed, validate file type and size (<5MB).

These rules typically translate to code validation (e.g. frontend rules, backend schema constraints).

Edge Cases

We enumerate unusual or extreme scenarios:

- **Empty Database:** How does the UI appear if no data exists yet? (Show a friendly empty-state message).
- **Maximal Load:** If user requests an extremely large report, the system should handle or limit the response.
- **Partial Failures:** If sending notification (email) fails, the core data operation should still complete, with a retry log.
- **Time Zone Handling:** If users are in different time zones (if applicable), store times in UTC and display appropriately.
- **Concurrent User Register:** If two users sign up with same email at nearly same time, one should fail with duplicate error.

Documenting these ensures testers create scenarios to cover them.

Technical Architecture

We propose a modular architecture with clear separation of concerns. For example:

- **Frontend:** A single-page application (SPA) or mobile app in React.js/Angular/Vue or Flutter.
- **Backend/API:** RESTful API server (Node.js + Express, Django/Python, or Java Spring) handling business logic.
- **Database:** Relational (PostgreSQL/MySQL) or NoSQL (MongoDB) depending on data complexity.
- **Machine Learning (if needed):** A separate service or microservice, possibly in Python (TensorFlow/PyTorch), accessed via API.
- **Message Broker:** (Optional) like Redis Pub/Sub or RabbitMQ for background tasks (emails, notifications).
- **External Services:** Integration points (e.g. payment gateway, SMS service, government API).

flowchart LR

```
A[User Interface (Web/Mobile)] -->|HTTP/HTTPS| B[API Server]
B --> C[Application Logic]
C --> D[(Database)]
C --> E[ML/AI Module]
C --> F[Authentication Service]
C --> G[External APIs]
E --> D
B --> H[Cache/Redis]
```

This illustrates a typical **3-tier architecture** plus auxiliary services. The frontend sends requests to the backend API, which enforces business rules and interacts with the database or ML component. We use HTTPS for all communication (TLS termination at server). By separating an ML module, we can independently train models and call them as needed (e.g. predict outcomes from inputs).

Comparison of Technology Stack Options

We evaluate popular options and recommend a pragmatic stack:

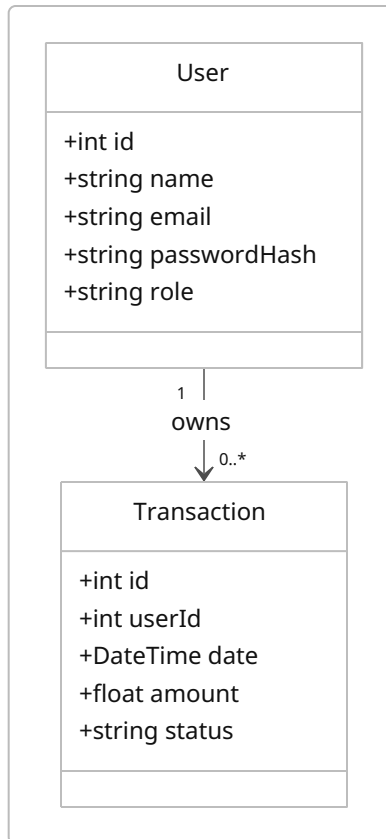
Layer/ Component	Option A	Option B	Option C	Recommendation
Frontend	React.js (JavaScript) – Popular SPA library with vast ecosystem.	Angular (TypeScript) – Full framework with strong structure.	Vue.js (JavaScript) – Lightweight progressive framework.	React or Vue: Fast dev, large community. E.g. React for flexibility 【4†L1501-L1504】.
Mobile	Flutter (Dart) – Cross-platform native performance.	React Native (JS) – Reuse React skills; good for native apps.	Ionic (JS/HTML) – Hybrid approach.	React Native if code- sharing with web; Flutter if starting fresh app.

Layer/ Component	Option A	Option B	Option C	Recommendation
Backend	Node.js + Express – Event-driven, good for JSON APIs.	Python + Django/Flask – Mature, lots of libs (esp. for ML).	Java + Spring Boot – Enterprise, highly scalable.	Node.js for quick prototyping; Python if heavy ML/data work [4†L1501-L1504] .
Database	PostgreSQL – Relational, ACID, supports JSON.	MongoDB – NoSQL, flexible schema, good for unstructured data.	MySQL – Relational, widespread.	PostgreSQL for structured data with relations; MongoDB if schema-evolution needed.
Authentication	JWT (JSON Web Tokens) – Stateless auth.	OAuth2 (Keycloak/ Auth0) – Robust auth server.	Custom session store – simpler for small team.	JWT + OAuth2 provider : JWT for API auth, possibly use Auth0/Keycloak for user management.
Hosting/Infra	AWS (EC2, RDS, Lambda, etc) – Most services, flexible.	Azure or Google Cloud – Similar cloud IaaS/PaaS offerings.	On-premise/VPS – If needed by constraints.	AWS/GCP : Convenient services (S3, Lambda, RDS). CI/CD via GitHub Actions or GitLab.
DevOps	Docker + Kubernetes – Container orchestration.	GitHub Actions/ Jenkins – CI/CD pipelines.	Terraform/ Ansible – IaC for infra.	Docker + GitHub Actions : Containerize and automate builds/ tests/deloys easily.
Monitoring	Prometheus + Grafana – Metrics and dashboards.	NewRelic/ Datadog – Commercial monitoring.	ELK stack (Elasticsearch/ Logstash/ Kibana) – Log aggregation.	Prometheus/Grafana + ELK : Open-source monitoring and logging.

Each choice has trade-offs. For a hackathon MVP, using managed services (e.g. Firebase for auth and DB, or AWS Amplify) could speed up development. However, for demonstrating mastery, a custom stack (e.g. Node/React/AWS) is often chosen. We recommend selecting the stack based on team expertise, but ensure it supports all functional needs with good community support.

Data Schema (Example)

A simplified entity-relationship diagram (ERD) or NoSQL schema example helps align the team. For instance, if the problem involves users and transactions:



This UML-like diagram shows entities and relationships. We would flesh out all entities (e.g. `Item`, `Order`, `Alert`, etc.) based on the problem domain.

API Contract (Example)

We list major API endpoints with methods and inputs/outputs:

- **POST /api/login:** `{ email, password }` -> returns `{ token, role }` or error.
- **GET /api/users/:id:** Returns user details (admin only).
- **POST /api/data:** Create a new data record (body: JSON fields) -> returns success or error.
- **GET /api/data?userId=X:** List data for a user.
- **PUT /api/settings/threshold:** Admin updates a threshold value.
- **POST /api/alert:** (Internal) triggers an alert based on input data.

Document each API with request/response schemas (e.g. via Swagger/OpenAPI spec or a markdown table). Include authentication (e.g. require `Authorization: Bearer token` for protected endpoints).

Integration Points

We note external integrations:

- Payment gateways (if payments involved).
- SMS/Email services (e.g. Twilio, SendGrid).

- Government or third-party APIs (if problem uses external data).
We ensure these are abstracted (e.g. via interface or service class) so they can be mocked in tests.

UI/UX Wireframes and Components

We outline the user interface structure and components.

Main Screens

1. **Login / Landing Page:** Fields for email/password, “Forgot password” link. Possibly social login buttons if needed.
2. **Dashboard** (for each role, different content):
3. **User Dashboard:** Quick summary cards (e.g. stats, latest records), navigation to main features (forms, history).
4. **Admin Dashboard:** Administrative overviews (user count, system status), links to user management.
5. **Data Entry Form:** A form page with labeled input fields, tooltips, and validation. Buttons: Save/Reset.
6. **User Management Screen:** Admin table of users with actions (Edit, Deactivate).
7. **Reports/Analytics Page:** Charts or tables (e.g. using a chart library like Chart.js).
8. **Settings/Profile Page:** User can update their profile or preferences.
9. **Error/Not Found Pages:** User-friendly 404 and 500 pages.

UI Components

- **Header/Nav Bar:** Logo, main navigation links, user avatar/dropdown for logout.
- **Sidebar** (optional): Menu for navigating sections (especially in admin panel).
- **Forms:** Standardized input components (text, dropdowns, datepicker). We mention frameworks (e.g. Material-UI if using React, or Bootstrap components).
- **Tables:** Sortable/searchable tables for data lists (e.g. user list).
- **Buttons & Modals:** Confirm modals (e.g. “Are you sure you want to delete?”).
- **Notifications/Toasts:** Feedback messages for user actions.
- **Loading Indicators:** Spinners or skeletons while data loads.

A wireframe mockup (e.g. rough diagram of the dashboard) can be created using any tool. (As an example, one could use an online template; here we describe it textually.) For instance, a dashboard might use a 12-column grid, with cards across the top showing KPIs and a data table below.

UX Considerations

- **Navigation Clarity:** Keep it simple; use breadcrumb or highlight current section.
- **Consistency:** Same look-and-feel and component styles across screens.
- **Mobile Responsiveness:** For SPAs, ensure grid reflows. Possibly a mobile-specific menu (hamburger).
- **Accessibility:** Sufficient contrast, ARIA labels on form inputs.

No-fluff advice: Prioritize usability testing of the UI early. A rough mockup should be validated with at least one user or mentor to catch obvious issues (since SIH judges often penalize poor UX).

Test Plan

Testing ensures quality at every level.

Unit Tests

- **Scope:** Test individual functions or classes (e.g. input validation functions, API controllers).
- **Examples:** Test that password hashing and verification works, that business rules trigger correct errors.
- **Frameworks:** Jest/Mocha for JavaScript, PyTest/Unittest for Python.
- **Coverage Goal:** Aim for high coverage on critical logic (e.g. 80%+ on core modules).

Integration Tests

- **Scope:** Test interactions between modules (e.g. API <-> Database, UI <-> API).
- **Examples:** An API test that creates a record, then fetches it from the database to confirm it was saved correctly.
- **Tools:** Use in-memory or test databases; Postman/Newman or supertest for REST endpoints.

End-to-End (E2E) Tests

- **Scope:** Simulate real user scenarios end-to-end.
- **Examples:** Using Selenium or Cypress: Register a user, log in, create data, verify it shows on dashboard.
- **CI Integration:** Automate E2E on each pull request or nightly build to catch regressions.

Test Data and Fixtures

- Prepare sample datasets covering normal and edge-case values.
- Include scripts to seed or reset the database for testing (e.g. factory fixtures for users, etc.).

Continuous Testing

- Tests run in CI (e.g. GitHub Actions) on every commit. Branch merges are gated on passing tests.
- Unit tests should be fast (seconds), while E2E can be nightly if long (to fit hackathon schedule).

Deployment, CI/CD, Security, and Monitoring

Deployment Pipeline

We recommend an automated CI/CD pipeline:

1. **Code Repository:** Use Git (GitHub/GitLab). Enforce feature branches and pull-requests with code review.
2. **Build & Test:** On push/PR, CI runs linters and all tests. Green build required to merge.

3. **Deployment:** After merge to main, trigger automatic deployment to a staging environment (e.g. via Docker and Kubernetes or serverless functions).
4. **Manual Verification:** Team checks staging, then manually approve production deploy.
5. **Production Deploy:** Use blue-green or rolling updates to avoid downtime.

Tools: GitHub Actions or Jenkins for CI; Docker for containerizing; Kubernetes/ECS for orchestration. Alternatively, use managed pipelines (GitLab CI/CD, AWS CodePipeline).

Security Checklist

- **Authentication/Authorization:** Use proven libraries for user auth. Apply “least privilege” on database access.
- **Encryption:** Encrypt data in transit (HTTPS) and sensitive data at rest (e.g. AES for PII). Use environment variables for secrets.
- **OWASP Top 10:** Defend against injection (use prepared statements/ORM), XSS (escape UI output), CSRF (tokens), etc. Follow OWASP guidance in coding and config.
- **Dependencies:** Scan for vulnerabilities (e.g. `npm audit`, `safety` for Python). Keep libraries up-to-date.
- **Logging/Audit:** Log security-relevant events (login attempts, config changes) to a secure location.
- **Incident Response:** Define a simple plan (who to notify, how to roll back) if a breach or critical bug occurs.

Monitoring and Logging

- **Application Logs:** Centralize (e.g. Elasticsearch/Logstash/Kibana) for error tracking. Tag logs with correlation IDs.
- **Metrics:** Use Prometheus to collect metrics (CPU, request latency, error rates). Dashboard with Grafana for real-time monitoring.
- **Alerts:** Set alerts (Slack/email) for critical issues (e.g. 500 errors spike, DB connectivity lost).
- **Health Checks:** Deploy an endpoint `/health` that checks service status (CPU, DB connection) for orchestration to use.

A sample monitoring architecture: user requests → API → logs to Elasticsearch; metrics to Prometheus; visualized in Kibana/Grafana; alerts via PagerDuty/Slack.

Data Privacy and Compliance

Given that SIH solutions often handle sensitive data, we note:

- **User Consent:** If personal data is collected, show privacy policy and obtain consent. Store consent records.
- **GDPR/India's PDP (if applicable):** Allow users to request data deletion or export. Avoid storing unnecessary PII.
- **Pseudonymization/Anonymization:** If analytics are done, strip identifiers. For example, replace user names with IDs in aggregated reports.
- **Secure Defaults:** Disable any data that should not be public by default (e.g. API data dumps require auth).

- **Compliance Standards:** If relevant, follow ISO 27001 or similar guidelines for data security. (SIH may not require formal certification, but awareness is good.)

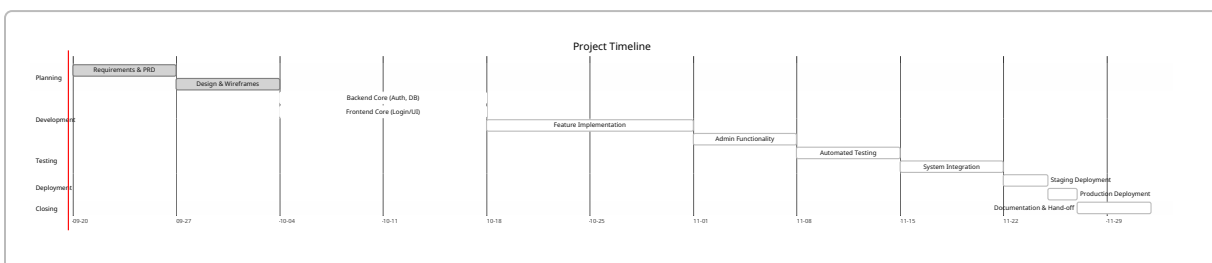
We do not hardcode any real personal data. Environment variables and key management should follow best practices.

Project Plan, Timeline, and Risk Register

High-Level Milestones

We break down the project into phases:

- **Phase 1 (Weeks 1-2):** Requirements & Design
 - Confirm requirements, finalize PRD, create initial designs/wireframes.
 - Set up dev environment and boilerplate code.
- **Phase 2 (Weeks 3-4):** Development Sprint 1
 - Implement authentication and core backend (APIs, DB schema).
 - Build basic frontend layout and login flow.
- **Phase 3 (Weeks 5-6):** Development Sprint 2
 - Add main functional features (data entry, processing).
 - Implement admin screens and basic reporting.
- **Phase 4 (Week 7):** Integration & Testing
 - Integrate frontend with backend, ensure end-to-end flows.
 - Complete unit/integration tests.
- **Phase 5 (Week 8):** UAT & Polish
 - Conduct user acceptance testing, fix bugs, improve UI.
 - Finalize documentation and demo scripts.
- **Phase 6 (Week 9):** Deployment & Presentation Prep
 - Deploy production environment, do security audit.
 - Prepare presentation/demo; ensure all deliverables complete.



Resource Estimates

- **Team:** 4-6 members (e.g. 2 backend devs, 1 frontend dev, 1 ML dev, 1 UI/UX, 1 QA).
- **Effort:** ~200 person-hours total (actual depends on scope). Plan sprints with 2-week iterations.
- **Tools:** Project management via Jira/Trello to track tasks and burndown. Version control on GitHub.

Roles & Responsibilities

Role	Responsibilities
Team Lead/PO	Stakeholder liaison, define scope, approve designs, backlog prioritization.
Backend Dev	Implement API, database schema, business logic, data model.
Frontend Dev	Build UI components, integrate APIs, ensure responsiveness.
UI/UX Designer	Create wireframes, ensure usability and consistent styling.
ML Engineer	(If needed) Develop/train model, expose inference API.
Tester/QA	Write and run test cases, report bugs, verify fixes.
DevOps Engineer	(Shared) Set up CI/CD pipelines, manage infrastructure, ensure deployments.

Each team member should clearly know their tasks. Regular scrum meetings (daily standups, sprint planning) keep everyone aligned.

Risk Register

Risk	Impact	Probability	Mitigation
Ambiguous requirements	High	Medium	Clarify with problem statement owner, freeze scope early.
Over-ambitious scope (feature creep)	High	High	Use MoSCoW to cut features, focus on MVP 【16†L30-L39】 .
Technical blocker (new tech)	Medium	Medium	Choose familiar tech stack; spike prototyping if uncertain.
Data privacy issues	High	Low	Keep PII minimal, consult advisor on compliance.
Integration delays (external API)	Medium	Medium	Mock external services; have backup test data.
Team member unavailability	Medium	Low	Cross-train team; maintain shared documentation (this PRD).
Insufficient testing time	High	Medium	Automate early; start testing alongside development.

For each risk, we assign a lead to monitor it. We continuously update this register.

Backlog (Prioritized Feature List)

ID	Feature/Story	Description	Priority (MoSCoW)	Status
US1	User Login/Signup	Secure auth, session management.	Must	Planned
US2	Data Submission Form	Form for core data input.	Must	Planned
US3	View/Filter Data	List and filter submitted data.	Must	Defined
US4	Admin: Manage Users	Admin can add/disable users.	Should	Defined
US5	Dashboard/Reports	Visual summary (charts, stats).	Should	Defined
US6	Email Notifications	Notify users on certain events.	Could	Future
US7	Settings/Config Page	Adjust app settings (admin only).	Could	Future
US8	Multi-language Support	UI translations for [languages].	Won't	Deferred

This table would continue until all known features are listed. It is a living document.

README and Contributor Guide (File: README.md, CONTRIBUTING.md)

We provide a sample structure:

README.md (Example Content)

```
# [Project Name]

[One-liner describing the solution and PS context]

## Features
- List key features (from PRD).

## Getting Started
1. Clone repository
2. Install dependencies (`npm install` or `pip install -r requirements.txt`)
3. Set up environment variables (see `.env.example` for keys).
4. Run database migrations or `setup.sql`.
5. Start backend server (`npm start` or `python app.py`).
6. Start frontend (`npm run dev`).

## Usage
```

- Navigate to `http://localhost:3000` to access the app.
- Default admin credentials: `admin@example.com / password` (change on first login).

Testing

Run tests with:

npm test

or

pytest

This runs unit and integration tests.

Deployment

A quick guide to deploy (e.g. Docker commands or cloud setup).

Documentation

Link to PRD and other docs: (e.g. docs/ folder or wiki).

License

[Appropriate license]

CONTRIBUTING.md (Example Content)

Contributing Guidelines

Thank you for your interest in contributing. Please follow these steps:

- **Code Standards**: Use consistent style (e.g. Prettier for JS, PEP8 for Python).
- **Branches**: Develop in feature branches named `feature/\*` or `fix/\*`.
- **Pull Requests**: Open a PR against `main`. Describe changes and reference any issue.
- **Review**: At least one other team member must review and approve code before merge.
- **Tests**: Ensure new code has tests. CI will run tests on all PRs.
- **Code of Conduct**: Be respectful. (Optional: link to a code of conduct file).
- **Issues**: Report bugs or request features in the issue tracker. Label appropriately.

We follow [CONTRIBUTING.md format example].

This provides clear instructions for future team members or open-source contributors on how to set up, run, and contribute to the project.

Appendix: Charts and Tables

Roles & Responsibilities

Role	Team Member (TBD)	Responsibilities
Project Manager	Person A	Define scope, schedule, stakeholder liaison.
Product Owner	Team Lead (You)	Prioritize backlog, maintain PRD.
Backend Developer	Person B	API/database development, business logic.
Frontend Developer	Person C	UI components, integrate with APIs.
UI/UX Designer	Person D	Wireframes, UI design, usability.
QA/Tester	Person E	Test plan, execute tests, report bugs.
DevOps Engineer	(All/Rotating)	CI/CD pipeline, deployments, monitoring setup.

Technology Stack Comparison

See table above in Architecture section.

Prioritized Backlog

See Backlog table in Project Plan section.

Sources: We drew on industry best practices and authoritative guides. For example, Atlassian defines a PRD as aligning stakeholders on purpose and features [4†L1501-L1504] . Acceptance criteria are “clear, concise, testable statements” that outline when a user story is done [42†L1694-L1702] . We follow established UX principles: a *user journey* is a high-level scenario path across channels, whereas a *user flow* is a task-focused step sequence [46†L83-L91] [46†L123-L131] . Prioritization uses standard MoSCoW technique (Must/Should/Could/Won't) [16†L30-L39] and backlog ordering ensures highest-value items first [16†L80-L88] . Throughout, we adopt an Agile mindset: iterative development, continuous feedback, and test-driven delivery.

This comprehensive documentation should serve as a blueprint for developing, testing, and deploying the SIH solution. It balances thorough planning with adaptability – we avoid “over-spec’ing” beyond core needs, focusing on shipping the right product with minimal waste [4†L1501-L1504] [42†L1798-L1807] . Each

section here can be saved as a Markdown file (or converted to HTML) and together packaged for the team's reference.
